PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Computação · Arquitetura de Computadores III

Prof. Ricardo Carlini Sperandio · 1º/2026

Trabalho Prático 2 — Simulação de Pipeline e Hazards

Relatório Analítico

Integrantes: Iago Verciano Romeiro

1. Objetivo e ambiente de simulação

Este trabalho analisa, no simulador **Ripes** (arquitetura RISC-V, RV32IM), o comportamento de um pipeline de 5 estágios (IF, ID, EX, MEM, WB) executando um programa de produto escalar. Foram comparados dois processadores do Ripes: o **5-stage processor w/o forwarding unit** (com detecção de hazards, mas sem forwarding) e o **5-stage processor** (com detecção de hazards e forwarding). Em ambos foi usada a extensão M, necessária para a instrução `mul`.

Para facilitar a leitura: **RAW** (Read-After-Write) ocorre quando uma instrução precisa ler um registrador que uma instrução anterior ainda não escreveu. Uma **bolha (stall)** é um ciclo em que o pipeline insere um `nop` para esperar o dado ficar disponível.

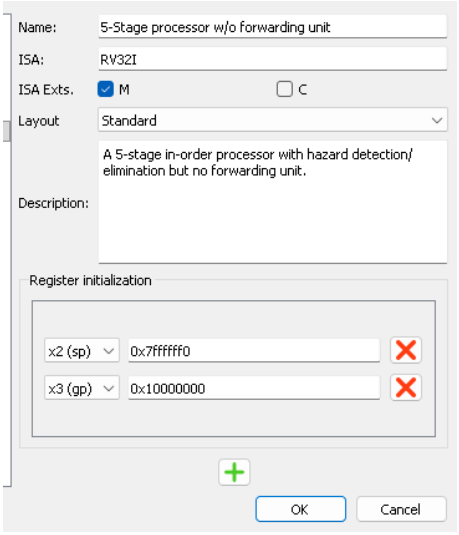


Figura 1a - Configuração do processador sem unidade de forwarding (Etapa 2).

2. Etapa 1 - Código Assembly

O programa `dot_product.s` calcula o produto escalar de dois vetores de 10 inteiros: A = {1, 2, ..., 10} e B = {10, 9, ..., 1}. Um laço percorre os vetores, multiplica A[i] por B[i] e acumula o resultado em `a0`. O resultado esperado é 1·10 + 2·9 + 3·8 + 4·7 + 5·6 + 6·5 + 7·4 + 8·3 + 9·2 + 10·1 = **220**.

```
# ============================================================
# TP2 - Simulação de Pipeline e Hazards (Arq. de Computadores III)
# Produto escalar (dot product) de dois vetores de inteiros
# Arquitetura: RISC-V (RV32IM) - simulador Ripes
#
# Resultado esperado em a0: 220
#   1*10 + 2*9 + 3*8 + 4*7 + 5*6 + 6*5 + 7*4 + 8*3 + 9*2 + 10*1
# ============================================================

.data
```

```
# Vetor A com 10 elementos (cada .word ocupa 4 bytes)
vetorA:   .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
# Vetor B com 10 elementos
vetorB:   .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

.text
.globl main
main:
    la   t0, vetorA        # t0 = ponteiro para o início de A
    la   t1, vetorB        # t1 = ponteiro para o início de B
    li   t2, 10            # t2 = contador de iterações (tamanho dos vetores)
    li   a0, 0             # a0 = acumulador do produto escalar (resultado final)

loop:
    lw   t4, 0(t0)         # t4 = A[i]
    lw   t5, 0(t1)         # t5 = B[i]
    mul  t6, t4, t5        # t6 = A[i] * B[i]
                           #   -> RAW em t5 (lw -> mul): load-use hazard
    add  a0, a0, t6        # acumula: a0 += t6
                           #   -> RAW em t6 (mul -> add)
    addi t0, t0, 4         # avança ponteiro de A para o próximo inteiro
    addi t1, t1, 4         # avança ponteiro de B para o próximo inteiro
    addi t2, t2, -1        # decrementa o contador
    bne  t2, zero, loop    # BRANCH DE FIM DE LAÇO: repete enquanto t2 != 0
                           #   -> RAW em t2 (addi -> bne) + hazard de controle

fim:
    li   a7, 10            # ecall 10 = encerrar o programa no Ripes
    ecall
```

Ao final da execução, o registrador `a0` (x10) contém `0x000000dc`, que é 220 em decimal, confirmando que o código está correto.



Figura 2 - Registrador a0 (x10) = 0xdc = 220 ao final da execução.

3. Etapa 2 - Pipeline SEM forwarding

Processador: **5-stage processor w/o forwarding unit**.

3.1 Métricas globais

Métrica	Valor
Número total de ciclos	176
Instruções executadas	88
CPI	176 ÷ 88 = 2
IPC	88 ÷ 176 = **0,5**

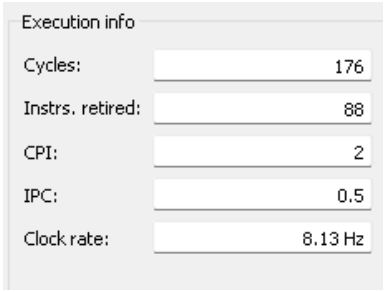


Figura 3 - Execution info sem forwarding: 176 ciclos, 88 instruções, IPC 0,5.

3.2 Momento 1: RAW em t5 entre lw e mul

No **ciclo 13** o `mul t6, t4, t5` está no estágio **ID**, onde precisa ler os registradores **t4 (x29)** e **t5 (x30)**. Esses valores são carregados pelos dois `lw` imediatamente anteriores, que neste ciclo ainda estão em EX (`lw t5`) e MEM (`lw t4`) e só escrevem o registrador no estágio **WB**. A dependência é, portanto, entre o estágio **WB** da instrução produtora (lw) e o estágio **ID** da consumidora (mul), com o `lw t5` sendo o mais recente e o que determina a espera.

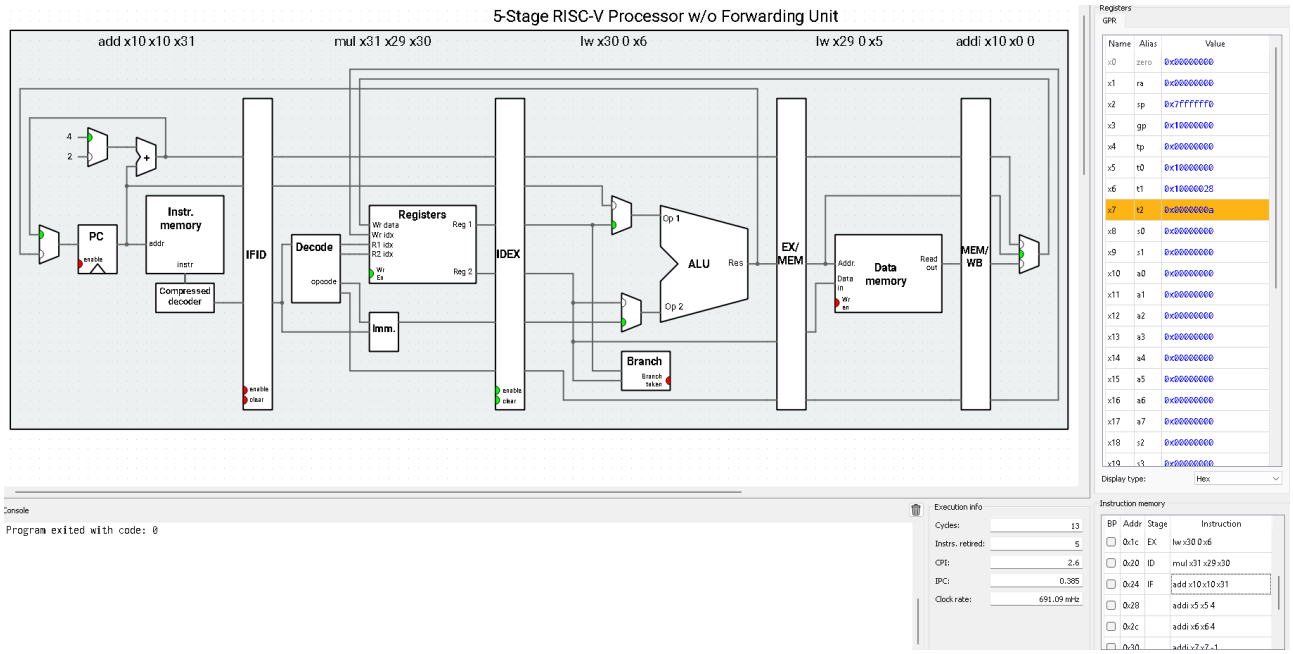


Figura 4 - Ciclo 13 (sem forwarding): add em IF, mul em ID, lw t5 em EX, lw t4 em MEM, addi em WB.

Como não há forwarding, o pipeline mantém o `mul` parado em ID e insere bolhas até o `lw t5` chegar ao WB. No **ciclo 16** o `mul` finalmente está em EX, com **2 bolhas (nop stall)** atrás dele, e o registrador t5 já vale 0x0000000a.

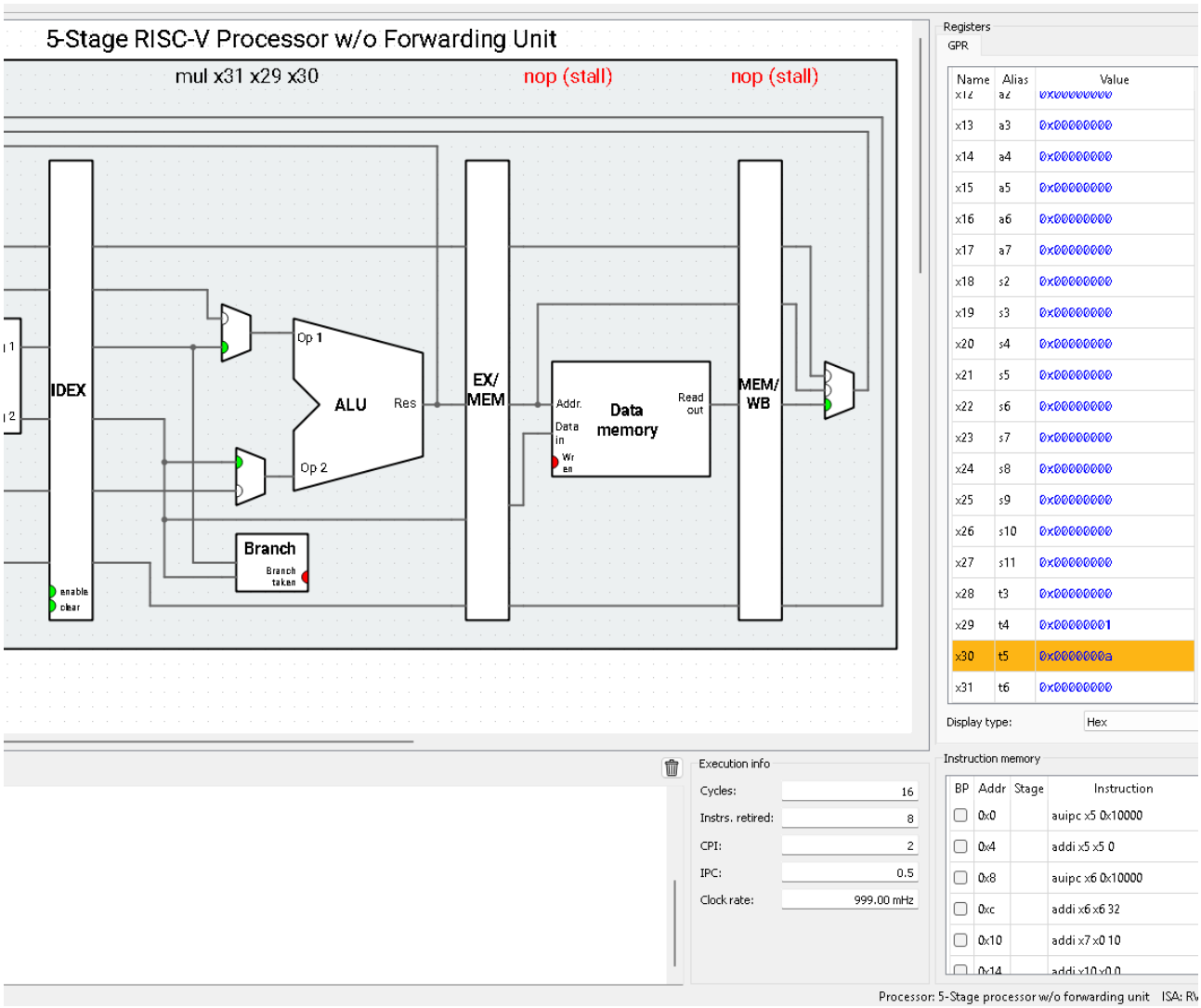


Figura 5 - Ciclo 16 (sem forwarding): mul em EX seguido de 2 nop (stall); t5 (x30) = 0xa já escrito.

3.3 Momento 2: RAW em t6 entre mul e add

No **ciclo 17** o `add a0, a0, t6` está em **ID** e precisa ler **t6 (x31)**, produzido pelo `mul` logo à frente, que está em MEM e ainda não escreveu o resultado (t6 continua em 0x00000000). O pipeline segura o `add` em ID e insere um `nop (stall)` em EX.

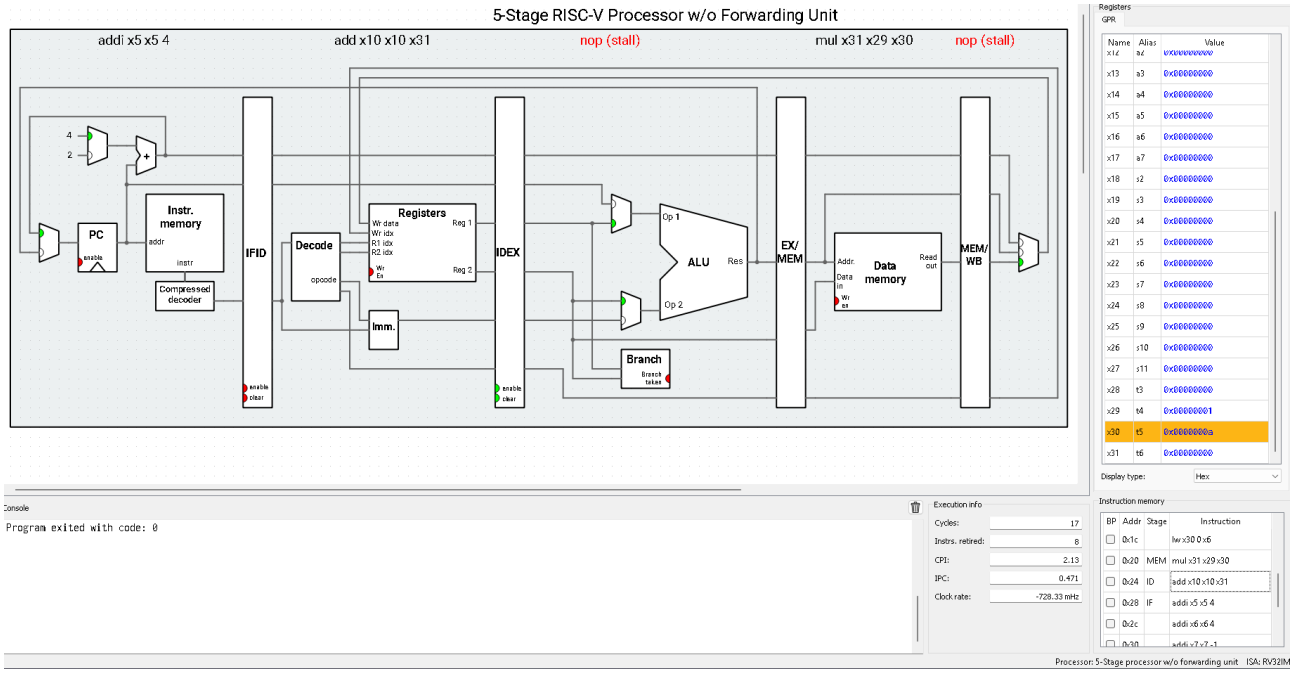


Figura 6 - Ciclo 17 (sem forwarding): add parado em ID, nop (stall) em EX, mul em MEM; t6 ainda é 0.

No **ciclo 18** o `mul` está em WB escrevendo o resultado, e as **2 bolhas** inseridas já aparecem em EX e MEM. Só no ciclo seguinte o add consegue ler t6 e avançar. A dependência é entre o estágio **WB** do mul e o estágio **ID** do add, e custa 2 ciclos.

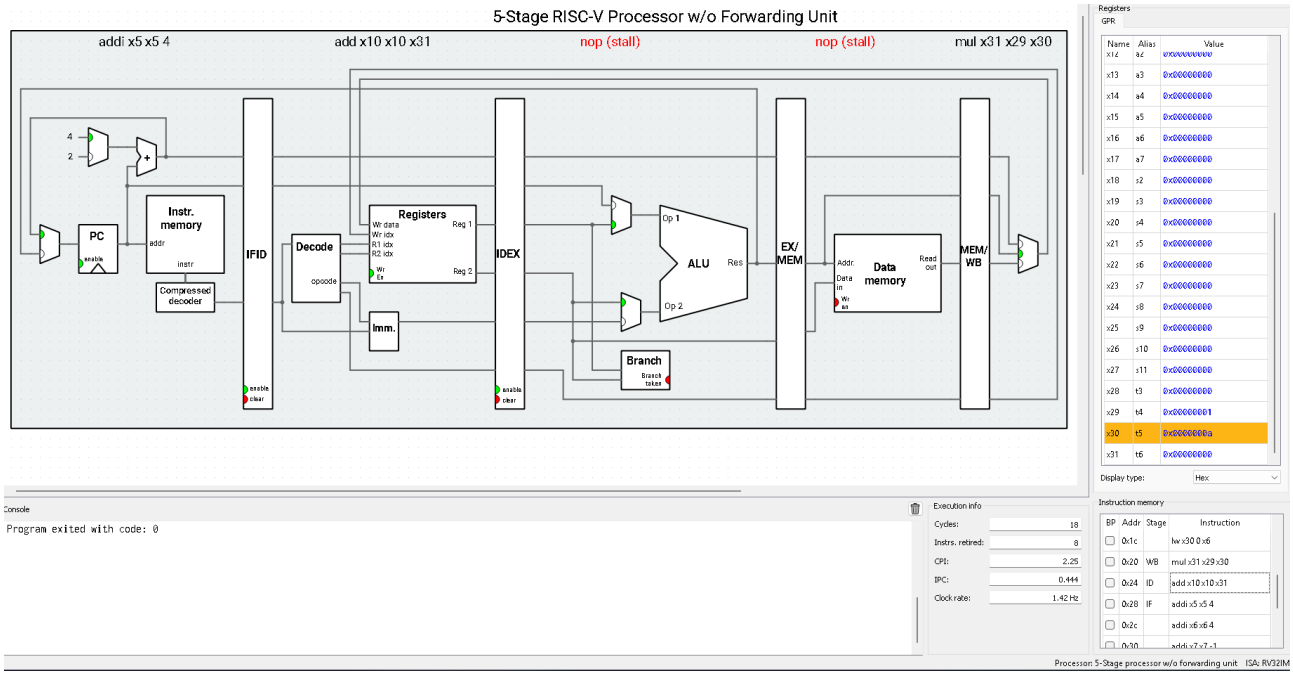


Figura 7 - Ciclo 18 (sem forwarding): add em ID com 2 nop (stall) à frente; mul em WB.

4. Etapa 3 - Pipeline COM forwarding

Processador: **5-stage processor** (hazard detection e forwarding), executando o mesmo código.

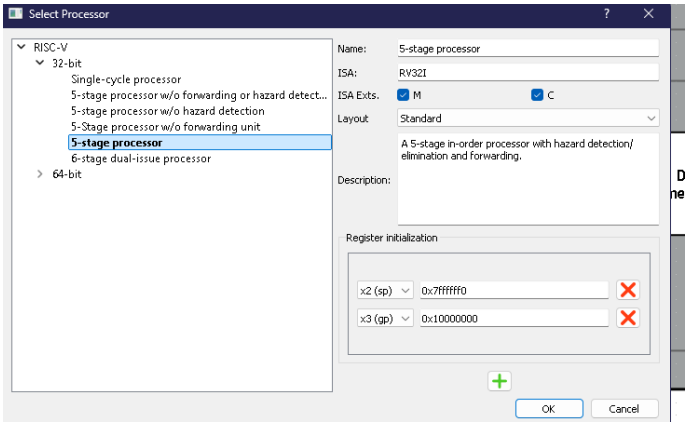


Figura 1b - Seleção do processador 5-stage processor (com hazard detection e forwarding).

4.1 Métricas globais e speedup

Métrica	Sem forwarding	Com forwarding
Ciclos	176	122
Instruções	88	88
CPI	2,00	1,39
IPC	0,500	**0,721** (88 ÷ 122)

Speedup = ciclos sem forwarding ÷ ciclos com forwarding = 176 ÷ 122 = 1,44. A simples adição do hardware de forwarding tornou o programa cerca de 44% mais rápido. O resultado continua correto (a0 = 0xdc).

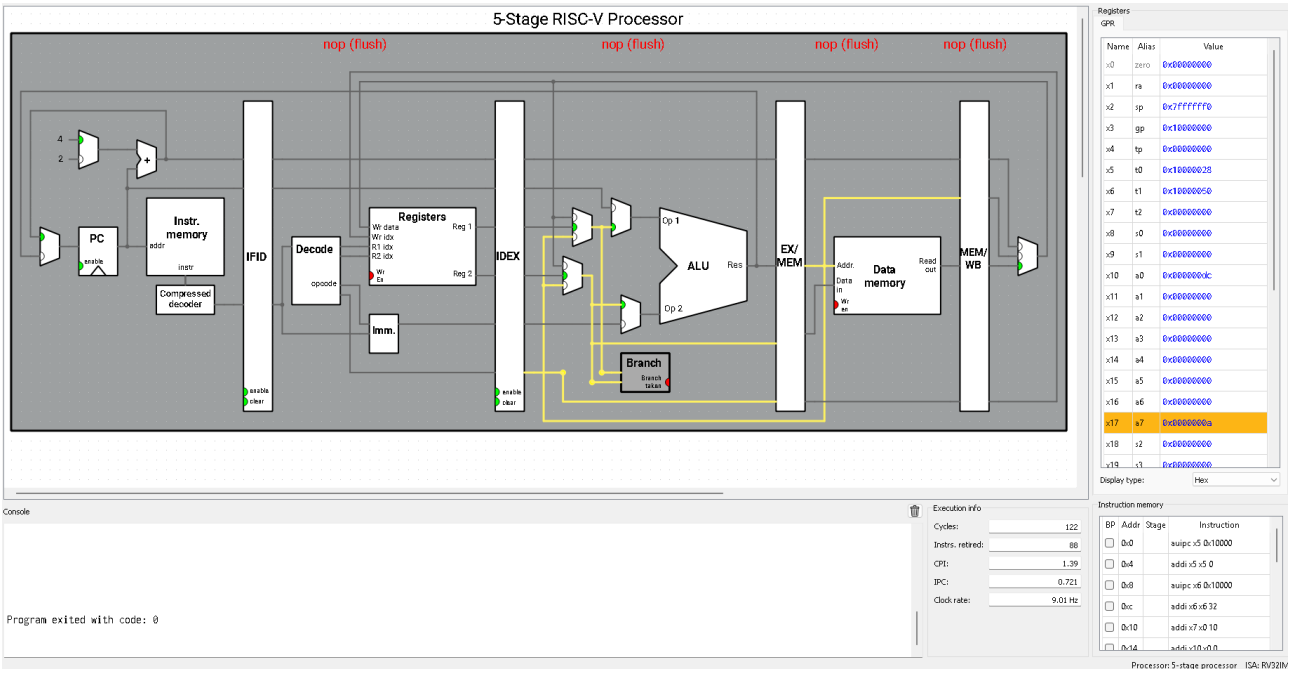


Figura 8 - Execution info com forwarding: 122 ciclos, 88 instruções, IPC 0,721; a0 = 0xdc.

4.2 Momento 1 revisitado: lw t5 e mul

Com forwarding, o número do ciclo muda (o programa avança mais rápido desde o início, pois dependências como auipc/addi do `la` deixam de gerar stalls). Por isso as mesmas instruções aparecem em ciclos diferentes: aqui o `mul` está em ID no **ciclo 10**. Agora há **apenas 1 bolha** (antes eram 2). O valor de t4 é adiantado pelos fios amarelos, mas o dado do `lw t5` só existe depois do estágio MEM; como o mul vem logo em seguida, ele precisa esperar 1 ciclo. Esse é um **load-use hazard**, que o forwarding não elimina por completo.

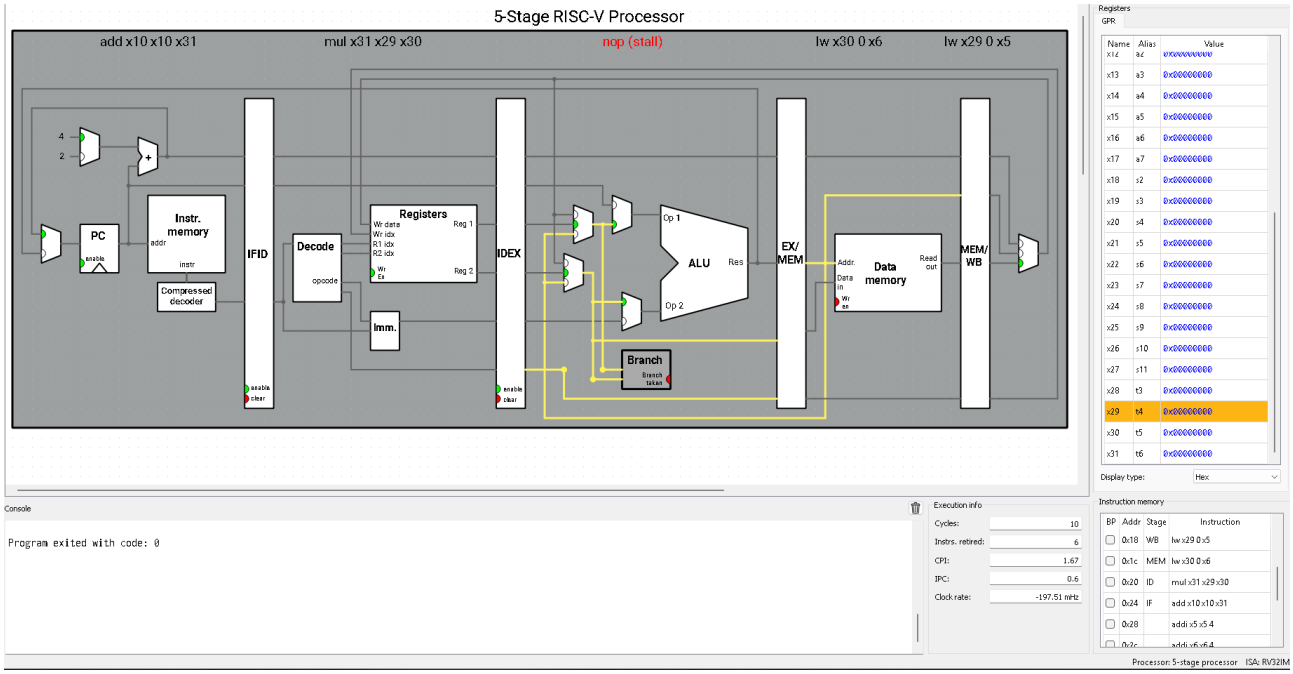


Figura 9 - Ciclo 10 (com forwarding): mul em ID, 1 nop (stall) em EX, lw t5 em MEM, lw t4 em WB.

No ciclo seguinte o `mul` vai para EX e recebe o valor de t5 pelo caminho de forwarding saindo do estágio MEM/WB para a entrada da ALU.

4.3 Momento 2 revisitado: mul e add

No **ciclo 12** o `add a0, a0, t6` está em EX **imediatamente atrás** do `mul`, que está em MEM, **sem nenhuma bolha entre eles**. O resultado do mul, que já está no registrador EX/MEM, é enviado de volta à entrada da ALU pelo fio amarelo (**de EX/MEM para a entrada da ALU**), sem esperar o WB. A bolha deste RAW **sumiu completamente** (de 2 para 0).

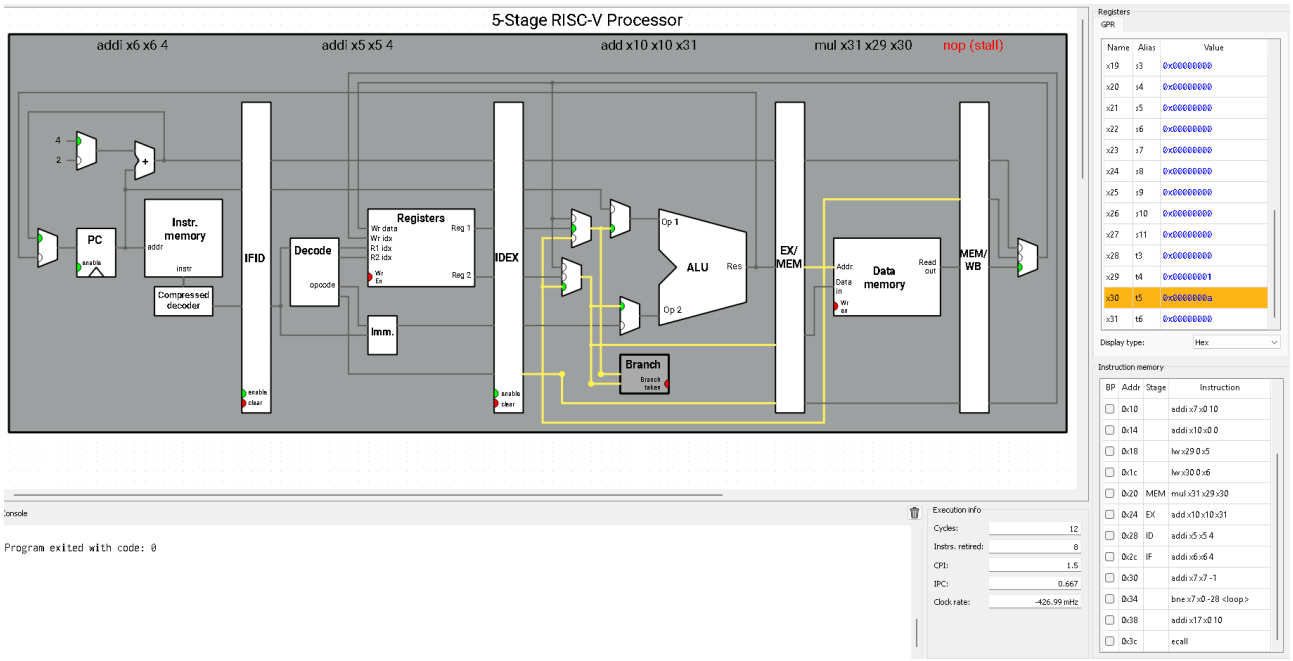


Figura 10 - Ciclo 12 (com forwarding): add em EX logo atrás de mul em MEM, sem bolhas; forwarding de EX/MEM para a ALU.

4.4 Comparação

Dependência	Sem forwarding	Com forwarding	Observação
lw t5 -> mul (RAW em t5)	2 bolhas	1 bolha	Sobrou load-use hazard: o dado do lw só fica pronto após MEM.
mul -> add (RAW em t6)	2 bolhas	0 bolhas	Resolvido por completo: forwarding de EX/MEM para a ALU.

Conclusão da etapa: a bolha do mul -> add sumiu completamente, mas sobrou 1 bolha de load-use no lw -> mul a cada iteração. Esse hazard só poderia ser evitado reordenando o código (colocando uma instrução independente entre o lw e o mul).

5. Etapa 4 - Hazards de controle

Branch do laço: `bne t2, zero, loop` (endereço 0x34), que repete o laço enquanto o contador t2 for diferente de zero.

Tratamento no simulador: o Ripes usa predição estática **predict not-taken**: o pipeline continua buscando as instruções seguintes ao branch e só descobre que ele foi tomado no estágio EX. No **ciclo 16** o `bne` está em EX com a luz *Branch taken* acesa (desvio tomado, pois t2 = 10), mas as duas instruções que o seguem no código (`addi a7, zero, 10` em ID e `ecall` em IF) já estavam no pipeline e são do caminho errado.

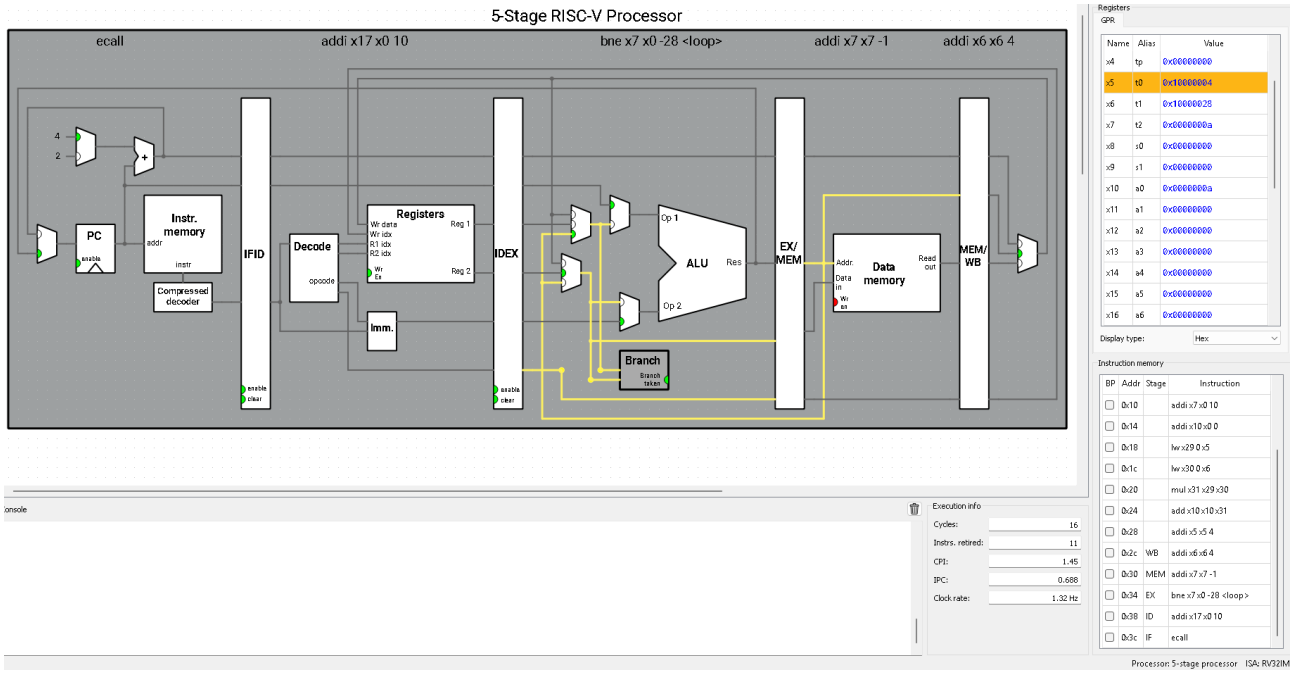


Figura 11 - Ciclo 16: bne em EX com Branch taken ativo; addi a7 (ID) e ecall (IF) são instruções erradas.

No **ciclo 17** essas duas instruções foram descartadas e aparecem como **2 nop (flush)**, em ID e EX, enquanto o PC já aponta para o início do laço (`lw t4` em IF).

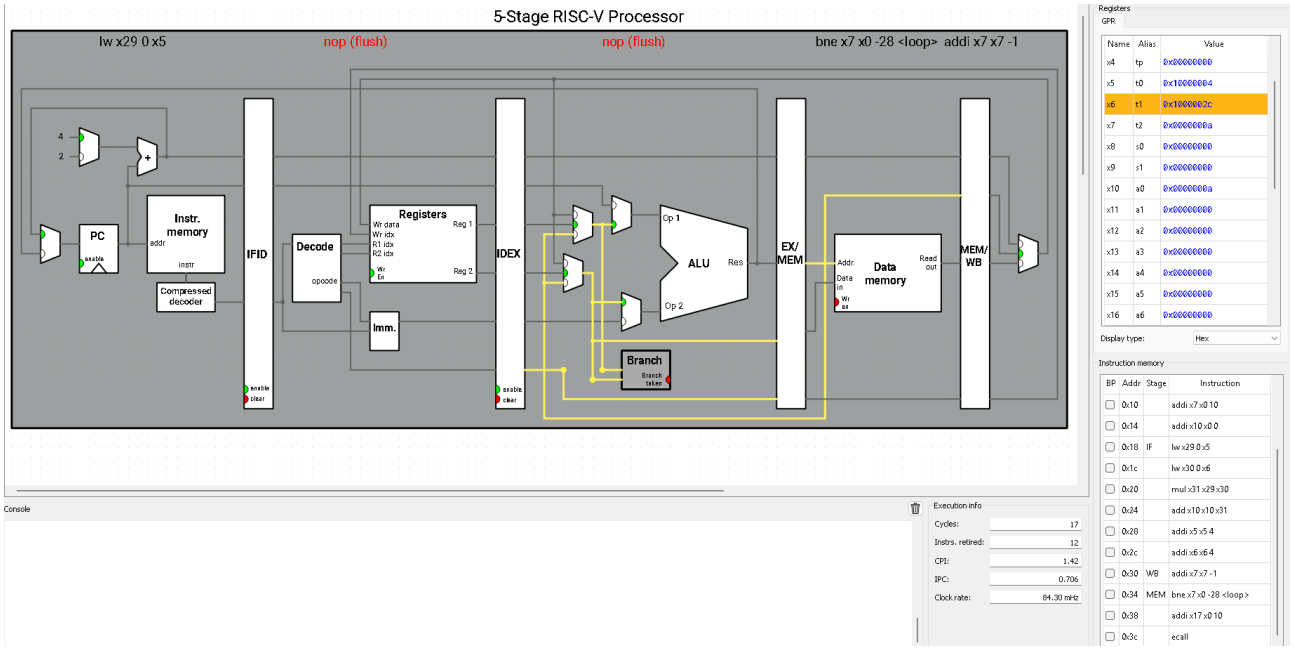


Figura 12 - Ciclo 17: 2 nop (flush) em ID e EX; lw t4 (início do laço) em IF.

Penalidade: 2 ciclos por branch tomado. O laço executa 10 iterações; o branch é tomado em 9 delas (na última, t2 chega a 0 e o laço termina), o que custa 9 × 2 = **18 ciclos** em flushes, cerca de 15% dos 122 ciclos totais. O forwarding não reduz esse custo, pois a penalidade vem da decisão do desvio, e não de dependência de dados.

6. Conclusão

Item	Resultado
Resultado do programa	a0 = 220 (0xdc)
Sem forwarding	176 ciclos, 88 instruções, IPC 0,500
Com forwarding	122 ciclos, 88 instruções, IPC 0,721
Speedup do forwarding	176 ÷ 122 = 1,44
Bolhas lw -> mul	2 sem forwarding, 1 com (load-use)
Bolhas mul -> add	2 sem forwarding, 0 com
Penalidade do branch tomado	2 ciclos (flush), 9 vezes = 18 ciclos

O forwarding é muito eficaz contra RAW entre instruções aritméticas, mas não resolve o load-use hazard nem a penalidade de desvios tomados. Para reduzir essas perdas seriam necessárias técnicas adicionais, como reordenação de instruções (para o load-use) e predição de desvios mais sofisticada (para o controle).